CVSS Score Extraction from Vulnerability Data

This skill provides guidance on extracting CVSS scores from vulnerability data—a critical component of securit
y report generation.

Overview

CVSS (Common Vulnerability Scoring System) provides a standardized way to assess the severity of security vuln
erabilities. When generating security reports, extracting the correct CVSS score from multiple data sources is
 essential for accurate risk assessment.

What is CVSS?

CVSS Scoring System

CVSS assigns a numerical score (0.0-10.0) representing vulnerability severity:

Score Range	Severity Level	Description
0.0	None	No impact
0.1-3.9	**Low**	Minimal impact
4.0-6.9	**Medium**	Moderate impact
7.0-8.9	**High**	Significant impact
9.0-10.0	**Critical**	Severe impact

CVSS Versions

- **CVSS v2**: Legacy scoring system (0-10 scale)
- **CVSS v3**: Current standard with refined metrics
- **CVSS v3.1**: Minor refinement of v3

Best Practice: Prefer CVSS v3/v3.1 scores when available.

Multiple Vulnerability Data Sources

Vulnerability scanners often aggregate data from multiple sources, each providing their own CVSS assessment:

Common Sources

1. **NVD (National Vulnerability Database)**
   - Maintained by NIST (U.S. government)
   - Most authoritative source
   - **Priority: Highest**

2. **GHSA (GitHub Security Advisory)**
   - Community-driven vulnerability database
   - Strong for open-source packages
   - **Priority: Medium**

3. **RedHat Security Data**
   - RedHat's security team assessments
   - Focused on enterprise Linux ecosystem
   - **Priority: Lower**

Why Multiple Sources?

- Not all sources have scores for every CVE

- Scores may differ based on interpretation
- Need fallback logic when primary source unavailable

Source Priority Strategy

When multiple sources provide scores, use a **priority cascade**:

```
NVD → GHSA → RedHat → N/A
```

Rationale: NVD is the most comprehensive and authoritative, followed by community sources, then vendor-specific databases.

Data Structure

Trivy (and similar tools) return CVSS data in nested format:

```json
{
  "VulnerabilityID": "CVE-2021-44906",
  "PkgName": "minimist",
  "Severity": "CRITICAL",
  "CVSS": {
    "nvd": {
      "V2Vector": "AV:N/AC:L/Au:N/C:P/I:P/A:P",
      "V3Vector": "CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H",
      "V2Score": 7.5,
      "V3Score": 9.8
    },
    "ghsa": {
      "V3Vector": "CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H",
      "V3Score": 9.8
    }
  }
}
```

Python Implementation

Basic Score Extraction

```python
def get_cvss_score(vuln_data):
    """
    Extract CVSS v3 score from vulnerability data.
    Uses priority: NVD > GHSA > RedHat

    Args:
        vuln_data: Dictionary containing vulnerability information

    Returns:
        CVSS v3 score as float, or 'N/A' if not available
    """
    cvss = vuln_data.get('CVSS', {})

    # Priority 1: NVD (National Vulnerability Database)
    if 'nvd' in cvss:
```

```
        score = cvss['nvd'].get('V3Score')
        if score is not None:
            return score

    # Priority 2: GHSA (GitHub Security Advisory)
    if 'ghsa' in cvss:
        score = cvss['ghsa'].get('V3Score')
        if score is not None:
            return score

    # Priority 3: RedHat
    if 'redhat' in cvss:
        score = cvss['redhat'].get('V3Score')
        if score is not None:
            return score

    # No score available
    return 'N/A'
```

Enhanced Version with V2 Fallback

```python
def get_cvss_score_with_fallback(vuln_data):
    """
    Extract CVSS score with v2 fallback.
    Priority: NVD v3 > GHSA v3 > RedHat v3 > NVD v2 > N/A
    """
    cvss = vuln_data.get('CVSS', {})

    # Try v3 scores first
    for source in ['nvd', 'ghsa', 'redhat']:
        if source in cvss:
            v3_score = cvss[source].get('V3Score')
            if v3_score is not None:
                return {'score': v3_score, 'version': 'v3', 'source': source}

    # Fallback to v2 if v3 not available
    if 'nvd' in cvss:
        v2_score = cvss['nvd'].get('V2Score')
        if v2_score is not None:
            return {'score': v2_score, 'version': 'v2', 'source': 'nvd'}

    return {'score': 'N/A', 'version': None, 'source': None}
```

Usage in Report Generation

Integrating CVSS Extraction

```python
import json

def parse_vulnerabilities(json_file):
    """Parse Trivy JSON and extract vulnerabilities with CVSS scores."""
    with open(json_file, 'r') as f:
        data = json.load(f)
```

```
    vulnerabilities = []

    if 'Results' in data:
        for result in data['Results']:
            for vuln in result.get('Vulnerabilities', []):
                # Extract basic fields
                record = {
                    'Package': vuln.get('PkgName'),
                    'Version': vuln.get('InstalledVersion'),
                    'CVE_ID': vuln.get('VulnerabilityID'),
                    'Severity': vuln.get('Severity'),
                    'CVSS_Score': get_cvss_score(vuln),  # Use extraction function
                    'Fixed_Version': vuln.get('FixedVersion', 'N/A'),
                    'Title': vuln.get('Title', 'No description')
                }
                vulnerabilities.append(record)

    return vulnerabilities
```

Common Patterns

Pattern 1: Numeric Score or 'N/A'

```python
cvss_score = get_cvss_score(vuln)
# Returns: 9.8 or 'N/A'
```

Use case: Simple reports where missing scores are acceptable

Pattern 2: Score with Metadata

```python
cvss_info = get_cvss_score_with_fallback(vuln)
# Returns: {'score': 9.8, 'version': 'v3', 'source': 'nvd'}
```

Use case: Detailed reports showing data provenance

Pattern 3: Filtering by Score Threshold

```python
def is_high_severity(vuln_data, threshold=7.0):
    """Check if vulnerability meets severity threshold."""
    score = get_cvss_score(vuln_data)
    if score == 'N/A':
        # If no score, use severity label
        return vuln_data.get('Severity') in ['HIGH', 'CRITICAL']
    return score >= threshold
```

Error Handling

Handling Missing Data

```python
def safe_get_cvss_score(vuln_data):
```

```
    """Safely extract CVSS score with comprehensive error handling."""
    try:
        cvss = vuln_data.get('CVSS', {})

        # Validate cvss is a dictionary
        if not isinstance(cvss, dict):
            return 'N/A'

        for source in ['nvd', 'ghsa', 'redhat']:
            if source in cvss and isinstance(cvss[source], dict):
                score = cvss[source].get('V3Score')
                # Validate score is numeric
                if score is not None and isinstance(score, (int, float)):
                    return score

        return 'N/A'
    except (AttributeError, TypeError):
        return 'N/A'
```

Best Practices

1. **Always provide fallback**: Use 'N/A' when scores unavailable
2. **Prefer newer versions**: V3 > V2
3. **Respect source hierarchy**: NVD is most authoritative
4. **Validate data types**: Ensure scores are numeric before using
5. **Document source**: In detailed reports, note which source provided the score

Complete Example

```python
import json

def get_cvss_score(vuln_data):
    """Extract CVSS v3 score with source priority."""
    cvss = vuln_data.get('CVSS', {})

    for source in ['nvd', 'ghsa', 'redhat']:
        if source in cvss:
            score = cvss[source].get('V3Score')
            if score is not None:
                return score

    return 'N/A'

def generate_report_with_cvss(json_file):
    """Generate vulnerability report with CVSS scores."""
    with open(json_file, 'r') as f:
        data = json.load(f)

    print(f"{'Package':<20} {'CVE ID':<20} {'CVSS':<8} {'Severity':<10}")
    print("-" * 60)

    if 'Results' in data:
        for result in data['Results']:
            for vuln in result.get('Vulnerabilities', []):
                pkg = vuln.get('PkgName', 'Unknown')
                cve = vuln.get('VulnerabilityID', 'N/A')
```

```
            cvss = get_cvss_score(vuln)
            severity = vuln.get('Severity', 'UNKNOWN')

            print(f"{pkg:<20} {cve:<20} {cvss:<8} {severity:<10}")

# Usage
generate_report_with_cvss('trivy_report.json')
```

Dependencies

Python Modules
- `json` (standard library)

Data Format
- Expects Trivy JSON format or similar structured vulnerability data

References

- [CVSS v3.1 Specification](https://www.first.org/cvss/v3.1/specification-document)
- [NVD CVSS Calculator](https://nvd.nist.gov/vuln-metrics/cvss/v3-calculator)
- [GitHub Security Advisories](https://github.com/advisories)